All-Pairs Shortest Paths with Fox's Algorithm

Min-plus repeated squaring distributed with MPI

Faculty of Sciences · University of Porto

Antonio Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Parallel Computing 2026/27
26 October 2026

Abstract. We solve the all-pairs shortest path problem by repeated squaring of the distance matrix under the min-plus semiring, with each product distributed over a $Q \times Q$ process grid by Fox's algorithm in C and MPI. On LabCC, four processes on one node reach a speedup of 3.9 on the assignment inputs and 4.0 on a random matrix with $N = 16384$. Once the grid spans several nodes, the 1 Gb/s network costs more than the extra processes gain, except on the largest random matrices.

1 Algorithm

The graph is an $N \times N$ matrix W, with INF for missing edges and 0 on the diagonal. The min-plus product replaces the sum by min and the product by $+$:

$$(A \otimes B)_{ij} = \min_k \big(A_{ik} + B_{kj}\big) \tag{1}$$

If D holds the shortest distances over paths of at most m edges, $D \otimes D$ holds them for $2m$ edges, so $\lceil \log_2(N-1) \rceil$ squarings of W give the answer, in $O(N^3 \log N)$. All versions stop early once a squaring leaves the matrix unchanged.

2 Implementation

One `src/main.c` is compiled as `seq`, `fox` (`-DUSE_MPI`) and `fox_shared` (`-DUSE_MPI -DFOX_SHARED`); the three share the I/O and the min-plus kernel. Distances are `uint32_t` in row-major order, with INF $= (2^{32}-1)/2$ so that adding two of them cannot overflow. Inputs are binary: N, then the N^2 entries.

2.1 Sequential version

`seq` times only `minplus_apsp`, which squares into a second buffer (the product cannot be computed in place) and swaps pointers:

```
D = W
for covered = 1; covered + 1 < N; covered *= 2:
    Next = D (x) D            # minplus_multiply_into
    if Next == D: break       # early termination
    swap(D, Next)
```

The kernel `minplus_relax` accumulates $C_{ij} \leftarrow \min\big(C_{ij}, A_{ik} + B_{kj}\big)$, so Fox can call it once per step on the same C. It runs the loops in i–k–j order, so the inner loop walks contiguous rows and vectorises into SIMD `min`; tiles them in 128×128 blocks to stay in cache; and skips row k of B when $A_{ik} = \text{INF}$.

2.2 Fox: scatter, (Irecv B, Isend B, Ibcast A)*, gather

Rank 0 checks that $P = Q^2$ and that Q divides N, builds a periodic Cartesian grid, and derives one communicator per row and per column. It packs the blocks of W in rank order, so one `MPI_Scatter` hands out the blocks and one `MPI_Gather` collects them at the end. Within each product, the transfers for step $s+1$ are started before step s computes, so a step costs about $\max\big(t_{\text{comp}}, t_{\text{comm}}\big)$:

```
MPI_Scatter W -> A;  B = A
repeat:                                          # one squaring
    MPI_Bcast A[i][i] along row i
    C = INF
    for s = 0 .. Q-1:
        if s + 1 < Q:
            MPI_Irecv  B_next from (i + 1) mod Q along column j
            MPI_Isend  B      to   (i - 1) mod Q along column j
            MPI_Ibcast A[i][(i + s + 1) mod Q] along row i -> A_next
        C = min(C, A_cur (x) B)                  # minplus_relax
        MPI_Waitall;  swap(B, B_next);  A_cur = A_next
    if not MPI_Allreduce(C != A, LOR): break     # early termination
    swap(A, C);  B = A
MPI_Gather A -> W
```

The computation only reads the buffers being sent, and the broadcast root sends straight from its own block. Each process holds six blocks: A, B, C, two A receive buffers and a spare B.

2.3 Shared copy: broadcast, (local product, `MPI_Allgather`)∗

`fox_shared` takes the communication out of the product. Every process keeps the whole matrix M, stored block by block in rank order, and reads A_{ik} and B_{kj} straight from it. After each squaring, one `MPI_Allgather` writes every new block into every copy; the rank-order layout puts rank r's block exactly where it belongs, with no packing:

```
MPI_Bcast M
repeat:
    C = INF
    for s = 0 .. Q-1:  k = (i + s) mod Q;  C = min(C, M[i][k] (x) M[k][j])
    if not MPI_Allreduce(C != M[i][j], LOR): break
    MPI_Allgather C -> M
```

Per squaring, `fox` moves $2N^2/Q$ values to each process and overlaps them with computation. `fox_shared` moves N^2, $Q/2$ times more, without overlap, and needs N^2 values of memory per process instead of $6(N/Q)^2$.

2.4 Validation

Unit tests cover I/O and the kernel, including all-INF matrices and a comparison with Floyd–Warshall. `seq` and `fox` match the reference outputs byte for byte for $N \in \{6, 300, ..., 3000\}$ with $P \in \{1, 4, 9, 16\}$, and invalid grids are rejected with the expected message.

3 Random inputs

The assignment inputs are not powers of two, so we generate matrices with `matgen N [count] [seed]`. The diagonal is 0, and every other entry is an edge with probability 10%, weighted uniformly in $[0, 2^{16}]$, and INF otherwise. Two `rand()` calls give 30 random bits per draw. A path of at most $N-1$ such edges stays below $1.07 \cdot 10^9 <$ INF for $N \le 2^{14}$. `make gen` writes $N = 2^6, ..., 2^{14}$ with seed 42.

Table 1. Edge density against squarings done (of $\lceil \log_2(N-1) \rceil$), unreachable pairs and `seq` time, on an AMD Ryzen 7 7840HS.

N	edges	squar.	unreach.	ms
64	100%	5 / 6	0%	0.2
64	10%	5 / 6	0%	0.1
64	1%	4 / 6	95.9%	0.0
2048	100%	6 / 11	0%	4996
2048	10%	6 / 11	0%	4572
2048	0.1%	6 / 11	33.1%	1477

Why 90% INF

At 10% edges every pair is still reachable for $N \geq 64$, while at 1% a 64-node graph leaves 96% of the pairs unconnected. The work is not made easier: 10% needs as many squarings as a complete graph and 92% of its time at $N = 2048$. And unlike a complete graph, it exercises INF handling and the kernel's skip (Table 1).

4 Performance evaluation

Times cover the computation only (scatter, squarings, gather), measured on rank 0 with `MPI_Wtime` after a barrier. We report the minimum of 3 runs on the assignment inputs and of 2 on the random matrices, with speedups $S = T_{\text{seq}}/T(P)$ against `seq` and $S_1 = T(1)/T(P)$ against `fox` with one process, and efficiency $E = S/P$. Each LabCC node has an AMD Ryzen 7 5700G (8 cores, 16 threads, 16 MB L3) and 14 GB of memory, runs Debian 12, and is connected by 1 Gb/s Ethernet; we used Open MPI 4.1.4 and GCC 12.2.

4.1 Assignment inputs on LabCC

All five assignment inputs fit $P \in \{1, 4, 9, 16, 25\}$. The hostfile lists 29 nodes with 8 slots each, filled in order, so $P = 4$ runs on one node, $P = 9$ and $P = 16$ on two, and $P = 25$ on four (`bench/report/lab_examples.csv`).

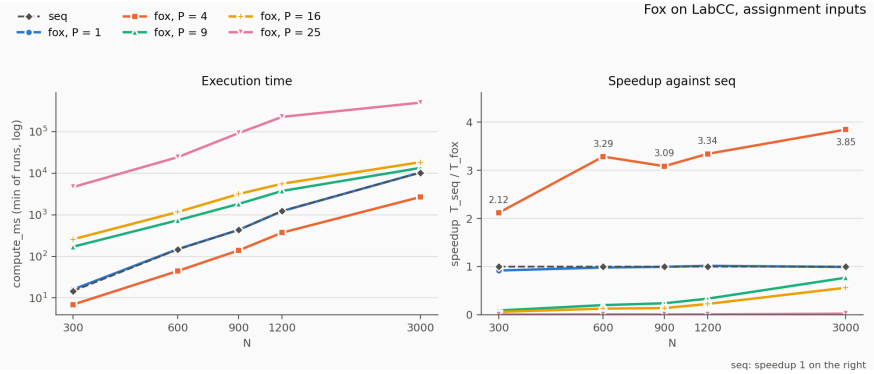


Figure 1. Time (log scale) and speedup on the assignment inputs on LabCC. Speedups below 0.5 are left unlabelled; they are in Table 3.

Table 2. Assignment inputs on LabCC: minimum of 3 runs in milliseconds.

N	seq	$P = 1$	$P = 4$	$P = 9$	$P = 16$	$P = 25$
300	14.7	16.0	6.9	170.2	257.5	4628.2
600	145.5	148.5	44.3	734.5	1159.8	24145.2
900	430.6	433.0	139.5	1820.5	3151.9	91880.0
1200	1238.9	1221.1	370.6	3729.3	5563.7	225073
3000	10249.0	10334.4	2663.4	13346.9	18320.4	494939

Table 3. Speedup S against `seq` and S_1 against `fox` with $P = 1$, from Table 2.

	$P = 4$		$P = 9$		$P = 16$		$P = 25$	
N	S	S_1	S	S_1	S	S_1	S	S_1
300	2.12	2.31	0.086	0.094	0.057	0.062	0.003	0.003
600	3.29	3.35	0.20	0.20	0.13	0.13	0.006	0.006
900	3.09	3.10	0.24	0.24	0.14	0.14	0.005	0.005
1200	3.34	3.29	0.33	0.33	0.22	0.22	0.006	0.005
3000	3.85	3.88	0.77	0.77	0.56	0.56	0.021	0.021

With $P = 1$, `fox` is within 2% of `seq` from $N = 600$ (9% slower on the 15 ms run at $N = 300$). On one node, four processes reach 3.1–3.9 from $N = 600$, with an efficiency of 0.96 at $N = 3000$. With 9, 16 and 25 processes, on two to four nodes, this run is slower than `seq` for every input, and $P = 25$ varies up to 3.7 times between runs. A rerun of $P = 9$ (Section 4.2) shows these multi-node times were inflated by other load on the shared nodes; we read them as upper bounds.

4.2 Placement of nine processes

Nine processes do not fit one 8-slot node. We reran $P = 9$ with the default placement (8 + 1) and three others (`bench/report/lab_p9_*.csv`, minimum of 3 runs; 2 for 2 + 2 + 2 + 2 + 1 at $N = 3000$).

Table 4. $P = 9$ on LabCC by placement: time in ms and speedup S against `seq`. "First run" is $P = 9$ from Table 2.

				8 + 1		**5 + 4**		**3 + 3 + 3**		**2+2+2+2+1**	
N	seq	$P = 4$	first run	time	S	time	S	time	S	time	S
300	14.7	6.9	170.2	26.8	0.55	35.8	0.41	28.8	0.51	303.8	0.048
600	145.5	44.3	734.5	96.3	1.51	183.1	0.79	141.3	1.03	1632.2	0.089
900	430.6	139.5	1820.5	288.7	1.49	375.5	1.15	334.4	1.29	2847.6	0.15
1200	1238.9	370.6	3729.3	636.7	1.95	770.7	1.61	709.2	1.75	5580.6	0.22
3000	10249.0	2663.4	13346.9	3118.2	3.29	4174.8	2.45	3745.5	2.74	24361.1	0.42

With the same 8 + 1 placement, the rerun is 4–8 times faster than the first run and beats `seq` from $N = 600$ (3.29 at $N = 3000$), but never beats four processes on one node. Ranks fill the grid row by row, so with 8 + 1 only the row and column of the remote process cross the 1 Gb/s link; with 3 + 3 + 3 every column shift of B does (8–47% slower); with 5 + 4 every column shift and one row broadcast do (21–90% slower); with 2 + 2 + 2 + 2 + 1 every row and column spans two nodes, so all transfers cross it (8–17 times slower).

4.3 Assignment inputs on an Apple M1 Max

For comparison we ran the same inputs on a MacBook with an Apple M1 Max (10 cores: 8 performance, 2 efficiency) and 32 GB of memory (`bench/report/mac_examples.csv`). With only 10 cores it can run $P =$ 1, 4 and 9, but not 16 or 25.

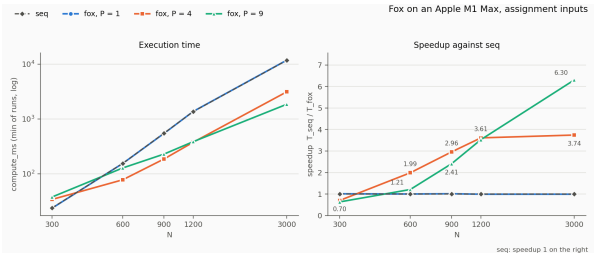


Figure 2. Time (log scale) and speedup on the M1 Max.

Table 5. M1 Max: minimum of 3 runs in ms, S against `seq`, S_1 against $P = 1$.

			$P = 4$			$P = 9$		
N	seq	$P = 1$	time	S	S_1	time	S	S_1
300	24.0	23.7	34.2	0.70	0.69	37.9	0.63	0.63
600	154.8	154.6	77.9	1.99	1.98	127.9	1.21	1.21
900	551.9	545.0	186.5	2.96	2.92	229.1	2.41	2.38
1200	1363.2	1373.9	377.5	3.61	3.64	386.3	3.53	3.56
3000	11658.2	11749.5	3114.7	3.74	3.77	1849.1	6.30	6.35

On one machine $P = 9$ avoids the network: it trails $P = 4$ up to $N = 1200$ (three steps per product, one process on an efficiency core) but reaches 6.30 at $N = 3000$.

4.4 Random matrices on LabCC

Here $P = 4$ runs on one node, $P = 16$ on two and $P = 64$ on eight; these are the process counts whose Q divides a power of two (`bench/report/lab_rand0.csv`).

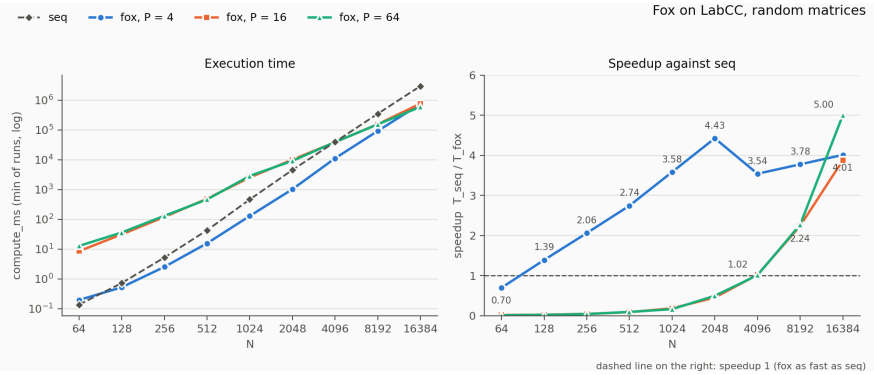


Figure 3. Time (log scale) and speedup on the random matrices on LabCC. Labels close to one already shown are omitted.

With four processes, efficiency stays between 0.89 and 1.11 from $N = 1024$. $P = 16$ and $P = 64$ take about the same time (within 15% from $N = 128$ to 8192), so the network, not the process count, sets it. They break even with `seq` at $N = 4096$ and reach 3.88 and 5.00 at $N = 16384$ ($P = 4$: 4.01).

4.5 Fox against the shared copy

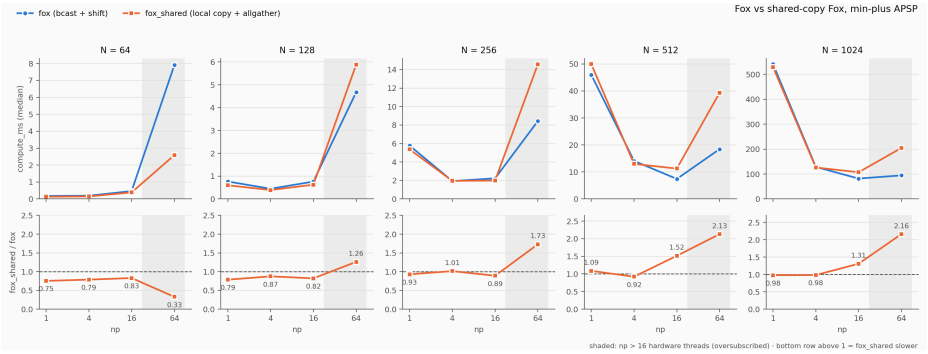


Figure 4. `fox` against `fox_shared`, median of 3 to 6 runs. Bottom: `fox_shared` time over `fox` time; shaded P oversubscribes the 16 hardware threads.

On an AMD Ryzen 7 7840HS laptop (`bench/report/results_power2.csv`, random matrices with $N = 2^6, ..., 2^{10}$), `fox_shared` is mostly faster, by up to 28%, for $N \leq 256$, where one all-gather beats $2Q$ latency-bound messages. From $N = 512$, at $P = 16$, it takes 1.3–1.5 times as long: it moves $Q/2 = 2$ times more data, without overlap. At $P = 4$ ($Q/2 = 1$) the two are within a few percent.

5 Conclusions

Fox scales well within a node: four processes reach a speedup of up to 3.9. Across nodes the 1 Gb/s link costs more than the extra processes gain, and only 64 processes beat four, at $N = 16384$. Overlapping the transfers keeps `fox` ahead of `fox_shared` on large matrices.